

医院预约挂号系统

实验工作记录和总结

一、项目基本信息

项目	内容
项目名称	医院预约挂号系统
开发周期	2025年12月1日 - 2025年12月31日
开发环境	Windows 11 + IntelliJ IDEA + VS Code
部署环境	腾讯云服务器 OpenCloudOS (152.136.13.236)
技术栈	Spring Boot 3.2 + Vue 3 + MySQL 8.0 + Docker
代码行数	约13,400行
访问地址	http://152.136.13.236:8081

二、实验工作记录

2.1 第一阶段：需求分析与设计（第1-2周）

表2-1 第一阶段工作记录

日期	工作内容	完成情况	遇到问题	解决方案
12/1-12/3	需求调研与分析	完成	功能范围不明确	参考现有挂号系统，确定核心功能
12/4-12/5	系统架构设计	完成	技术选型犹豫	对比Spring Boot和Django，选择前者
12/6-12/7	数据库设计	完成	表关系复杂	绘制ER图，理清实体关系
12/8-12/10	API接口设计	完成	RESTful规范不熟	学习REST最佳实践

2.2 第二阶段：后端开发（第3-5周）

表2-2 第二阶段工作记录

日期	工作内容	完成情况	遇到问题	解决方案
12/11-12/12	Spring Boot项目搭建	完成	依赖版本冲突	统一使用Spring Boot 3.2管理依赖
12/13-12/15	用户认证模块开发	完成	JWT配置复杂	参考官方文档配置Spring Security
12/16-12/18	预约挂号模块开发	完成	并发扣减号源问题	使用JPA乐观锁@Version解决
12/19-12/20	排班管理模块开发	完成	时间段逻辑复杂	设计Schedule和TimeSlot两级结构
12/21-12/22	科室管理模块开发	完成	无	-
12/23	AI智能助手开发	完成	API调用超时	设置合理的超时时间和重试机制

2.3 第三阶段：前端开发（第4-6周）

表2-3 第三阶段工作记录

日期	工作内容	完成情况	遇到问题	解决方案
12/14-12/15	Vue项目搭建	完成	Vite配置不熟	学习Vite官方文档
12/16-12/18	登录注册页面	完成	表单验证复杂	使用Element Plus表单验证
12/19-12/22	患者端界面开发	完成	响应式布局问题	使用CSS媒体查询适配
12/23-12/25	医生端界面开发	完成	日历组件选择	使用Element Plus日期选择器
12/26-12/27	管理员端界面开发	完成	权限路由控制	使用Vue Router导航守卫
12/28	AI助手界面开发	完成	打字机效果实现	使用定时器逐字显示

2.4 第四阶段：测试与部署（第7周）

表2-4 第四阶段工作记录

日期	工作内容	完成情况	遇到问题	解决方案
12/29	功能测试	完成	部分接口返回异常	修复后端代码逻辑
12/30	Docker配置	完成	容器网络通信问题	配置Docker Compose网络
12/30	服务器部署	完成	CORS跨域问题	配置Spring Security允许跨域
12/30	数据库迁移	完成	中文乱码问题	设置MySQL字符集为utf8mb4
12/31	系统联调	完成	医生列表为空	修复getTopDoctors查询逻辑
12/31	文档编写	完成	无	-

三、关键技术问题及解决方案

3.1 并发预约号源扣减问题

问题描述：多个用户同时预约同一时段时，可能出现超卖现象。

解决方案：使用JPA乐观锁机制

```
// TimeSlot实体添加版本号字段
@Version
@Column(nullable = false)
private Integer version = 0;

// 扣减号源时自动检查版本号
@Transactional
public void decrementSlot(String timeSlotId) {
    TimeSlot slot = timeSlotRepository.findById(timeSlotId)
        .orElseThrow(() -> new RuntimeException("时段不存在"));
    if (slot.getRemainingSlots() <= 0) {
        throw new RuntimeException("号源已满");
    }
    slot.setRemainingSlots(slot.getRemainingSlots() - 1);
    timeSlotRepository.save(slot); // 乐观锁自动检查
}
```

3.2 JWT认证与Spring Security集成

问题描述：Spring Security 6.x配置方式变化大，与JWT集成复杂。

解决方案：自定义JWT过滤器

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response, FilterChain chain) {
        String token = extractToken(request);
        if (token != null && jwtUtil.validateToken(token)) {
            String userId = jwtUtil.getUserIdFromToken(token);
            UsernamePasswordAuthenticationToken auth =
                new UsernamePasswordAuthenticationToken(userId, null, authorities);
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
        chain.doFilter(request, response);
    }
}
```

3.3 Docker部署CORS跨域问题

问题描述：前端容器访问后端容器时出现CORS错误。

解决方案：配置Spring Security允许跨域

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(Arrays.asList(
        "http://localhost:8081",
        "http://152.136.13.236:8081"
    ));
    config.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE"));
    config.setAllowCredentials(true);
    config.setAllowedHeaders(Arrays.asList("*"));
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

3.4 AI接口调用优化

问题描述：通义千问API响应较慢，用户体验差。

解决方案：前端实现打字机效果，后端设置超时

```
// 前端打字机效果
const typeWriter = (text, callback) => {
  let index = 0;
  const timer = setInterval(() => {
    if (index < text.length) {
      callback(text.substring(0, index + 1));
      index++;
    } else {
      clearInterval(timer);
    }
  }, 30);
};
```

四、项目成果统计

4.1 功能模块完成情况

表4-1 功能模块完成统计

模块	计划功能点	完成功能点	完成率
用户认证	8	8	100%
预约挂号	12	12	100%
排班管理	10	10	100%
科室管理	6	6	100%
AI智能助手	5	5	100%
消息通知	4	4	100%
前端界面	20	20	100%
合计	65	65	100%

4.2 代码统计

表4-2 代码行数统计

模块	文件数	代码行数	占比
后端Java代码	45	5,200	38.8%
前端Vue代码	35	6,500	48.5%
SQL脚本	5	800	6.0%
配置文件	10	500	3.7%
Docker配置	3	400	3.0%
合计	98	13,400	100%

4.3 测试结果统计

表4-3 测试结果统计

测试类型	测试用例数	通过数	失败数	通过率
模块测试	12	12	0	100%
功能测试	12	12	0	100%
性能测试	4	4	0	100%
兼容性测试	4	4	0	100%
安全测试	4	4	0	100%
合计	36	36	0	100%

4.4 项目文档清单

表4-4 项目文档清单

序号	文档名称	页数	完成日期
1	项目开发计划书	8	2025-12-31
2	设计规格说明书	12	2025-12-31
3	系统实现及测试报告	10	2025-12-31
4	系统图表汇总	8	2025-12-31
5	实验工作记录和总结	6	2025-12-31
6	部署文档(DEPLOY.md)	2	2025-12-31

五、实验总结

5.1 项目收获

5.1.1 技术能力提升

- 后端开发：深入掌握了Spring Boot 3.x框架，包括Spring Security、Spring Data JPA的使用，理解了RESTful API设计规范
- 前端开发：熟练使用Vue 3组合式API和Element Plus组件库，掌握了Pinia状态管理和Vue Router路由配置
- 数据库设计：学会了规范的数据库设计方法，包括ER图绘制、表结构设计、索引优化等
- 容器化部署：掌握了Docker和Docker Compose的使用，能够实现应用的容器化部署
- AI集成：学会了调用大语言模型API，实现智能问答功能

5.1.2 工程能力提升

- 项目管理：学会了使用增量迭代开发模型，合理分解任务，控制项目进度
- 文档编写：掌握了软件工程文档的编写规范，包括需求文档、设计文档、测试文档等
- 问题解决：培养了独立分析和解决技术问题的能力，学会了查阅文档和搜索解决方案
- 版本控制：熟练使用Git进行代码版本管理

5.2 存在不足

- 功能方面：暂未实现短信验证、在线支付等功能，系统功能还可进一步完善
- 性能方面：未进行大规模并发测试，系统在高并发场景下的表现有待验证
- 安全方面：安全措施相对基础，可以增加更多安全防护机制
- 移动端：虽然支持响应式布局，但移动端体验还可以进一步优化
- 测试覆盖：单元测试覆盖率不够高，可以增加更多自动化测试

5.3 改进方向

- 增加短信验证码功能，提升账户安全性
- 集成在线支付功能，支持挂号费在线支付

- 开发原生移动端APP，提升移动端用户体验
- 引入Redis缓存，提升系统性能
- 增加单元测试和集成测试，提高代码质量
- 实现消息推送功能，及时通知用户预约状态变化
- 增加数据统计分析功能，为医院运营决策提供支持

5.4 心得体会

通过本次医院预约挂号系统的开发，我深刻体会到了软件工程的重要性。一个完整的软件项目不仅仅是编写代码，更需要规范的需求分析、系统设计、测试验证和文档编写。

在开发过程中，我遇到了很多技术难题，如并发控制、跨域问题、容器部署等。通过查阅文档、搜索资料、反复调试，最终都找到了解决方案。这个过程虽然艰辛，但也让我收获了宝贵的经验。

前后端分离的开发模式让我对现代Web应用架构有了更深入的理解。Spring Boot和Vue的组合确实能够高效地开发出功能完善的Web应用。Docker容器化部署也大大简化了应用的部署和运维工作。

总的来说，这次实验让我将课堂上学到的理论知识应用到了实际项目中，不仅提升了技术能力，也培养了工程思维和解决问题的能力。这些经验将对我今后的学习和工作产生积极的影响。

完成日期：2025年12月31日

签名： _____